

Accessing Robot Arm Animations

__Accessing Robot Arm Animations

SimTalk provides the *methods* listed in the table of contents to the left for robot arm animations in 3D.

You can buffer the entirety or all animations of a type in a value of data type *any*, for example:

```
var a : any := .PickAndPlace._3D.RobotArmAnimations
```

[_3D.RobotArmAnimations.getTable \[SimTalk\]](#)

Returns the existing **Robot Arm Animations** data of the object designated by <Path> and writes it into the passed table.

Remarks

Plant Simulation automatically formats the passed table.

Type

Method

Syntax

```
<Path>._3D.RobotArmAnimations.getTable(Animations:table)
```

Parameter

The parameter *Animations* of data type *table* designates the table which contains the robot animations.

The table has three columns:

- The column *From* of data type *string* indicates at which object a robot arm animation starts.
It designates the default orientation as starting point if the cell is empty.
- The column *To* of data type *string* indicates to which object a robot arm animation moves.
It designates the default orientation as the destination if the cell is empty.
- The column *Table* of data type *table* indicates the robot arm animation path.
It is a subtable with three columns of data type *length* for the X, Y, and Z values of all coordinates which the robot is to pass on its route from the *From position* to the *To position* within the coordinate system of the robot.

Example

```
var t : table  
PickAndPlace1._3D.RobotArmAnimations.getTable(t)  
PickAndPlace2._3D.RobotArmAnimations.setTable(t)
```

See also

[_3D.RobotArmAnimations.setTable \[SimTalk\]](#)

[Edit 3D Properties \[dialog\]](#) > [Robot Arm Animation \[PickAndPlace\]](#)

[_3D.RobotArmAnimations.setTable \[SimTalk\]](#)

Sets the table that contains the **Robot Arm Animations** of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.RobotArmAnimations.setTable(Animations:table
```

Parameter

The parameter *Animations* of data type *table* designates the table which contains the robot animations.

The table has three columns:

- The column *From* of data type *string* indicates at which object a robot arm animation starts.
It designates the default orientation as the starting point if the cell is empty.
- The column *To* of data type *string* indicates to which object a robot arm animation moves.
It designates the default orientation as the destination if the cell is empty.
- The column *Table* of data type *table* indicates the robot arm animation <Path>.

It is a subtable with three columns of data type *length* for the X, Y, and Z values of all coordinates which the robot is to pass on its route from the *From position* to the *To position* within the coordinate system of the robot.

Example

```
var t : table  
PickAndPlace1._3D.RobotArmAnimations.getTable(t)  
PickAndPlace2._3D.RobotArmAnimations.setTable(t)
```

See also

[_3D.RobotArmAnimations.getTable \[SimTalk\]](#)

[Edit 3D Properties \[dialog\]](#) > [Robot Arm Animation \[PickAndPlace\]](#)

Accessing Joints

[__Accessing Joints](#)

SimTalk provides the *functions* listed in the table of contents to the left for accessing **joints**.

See also

[Joint](#)

[Poses](#)

[__Accessing Poses](#)

[_3D.JointUseShortestRotationDirection \[SimTalk\]](#)

Sets if the rotational joint of the object designated by <Path> uses the spatially shortest direction toward its target orientations (`true`) or not (`false`).

Remarks

If the joint uses the spatially shortest direction, you cannot specify joint limits.

Type

Attribute

Syntax

```
<Path>._3D.JointUseShortestRotationDirection:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyCrane._3D.getObject(1).JointUseShortestRotationDirection := true
```

See also

[Type of Joint](#) > Revolute Joint

[Use Shortest Rotation Direction](#)

[JointLowerLimitActive \[SimTalk\]](#)

[JointLowerLimit \[SimTalk\]](#)

[JointUpperLimitActive \[SimTalk\]](#)

[JointUpperLimit \[SimTalk\]](#)

JointAcceleration [SimTalk]

Sets the acceleration of the joint of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>.JointAcceleration:acceleration
```

Assignment Value

The specified value designates:

- The acceleration in meters per second squared for a **Prismatic Joint**.
For a **Prismatic Joint** you assign a value of data type *acceleration*.
- The acceleration in degrees per second squared for a **Revolute Joint**.
For a **Revolute Joint** you assign a value of data type *real*.

Example

```
MyStation._3D.getObject(1).JointAcceleration := 0.3
```

See also

[_3D.getObject \[SimTalk\]](#)

[Acceleration \[text box\] - joint](#)

[JointDeceleration \[SimTalk\]](#)

JointCurrentValue [SimTalk]

Returns the current joint value of the object designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>.JointCurrentValue -> length
```

Return Value

The specified value designates:

- Meters for a **Prismatic Joint**.
For a **Prismatic Joint** the return value has the data type *length*.
- Degrees for a **Revolute Joint**.
For a **Revolute Joint** the return value has the data type *real*.

Example

```
print MyStation._3D.getObject(1).JointCurrentValue
```

See also

[_3D.getObject \[SimTalk\]](#)

[JointVelocity \[SimTalk\]](#)

[JointLowerLimit \[SimTalk\]](#)

[JointUpperLimit \[SimTalk\]](#)

JointDeceleration [SimTalk]

Sets the deceleration of the joint of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>.JointDeceleration:acceleration
```

Assignment Value

The specified value designates:

- The acceleration in meters per second squared for a **Prismatic Joint**.
For a **Prismatic Joint** you assign a value of data type *acceleration*.
- The acceleration in degrees per second squared for a **Revolute Joint**.
For a **Revolute Joint** you assign a value of data type *real*.

Example

```
MyStation._3D.getObject(1).JointDeceleration := 0.3
```

See also

[Acceleration \[text box\] - joint](#)

[_3D.getObject \[SimTalk\]](#)

[Acceleration \[text box\] - joint](#)

[JointAcceleration \[SimTalk\]](#)

JointLowerLimit [SimTalk]

Sets the lower limit of the joint of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>.JointLowerLimit:length
```

Remarks

Plant Simulation exclusively uses limits while **defining poses**. It does not use them to influence any other behavior, such as [_3D.Poses.moveToCoordinate \[SimTalk\]](#), [_3D.Poses.moveToMU \[SimTalk\]](#), etc.

Assignment Value

The specified value designates:

- Meters for a **Translation**.
For a **Translation** you assign a value of data type *length*.
- Degrees for a **Rotation**. [_3D.getObject](#) enables you to access animatable objects.
For a **Rotation** you assign a value of data type *real*.

Example

```
MyStation._3D.getObject(1).JointLowerLimit := 43
```

See also

[_3D.getObject \[SimTalk\]](#)

Lower Limit

JointLowerLimitActive [SimTalk]

Sets if the lower limit of the joint of the object designated by <Path> is active (**true**) or not (**false**).

Remarks

`_3D.getObject` enables you to access animatable objects.

Type

Attribute

Syntax

```
<Path>.JointLowerLimitActive:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Specify **true** to activate the limit if it has not been activated before. It will be activated with the other active limit if the other limit (upper/lower) has a value. It will be activated with 0 if no value was assigned.

Examples

```
MyStation._3D.getObject(1).JointLowerLimitActive
```

```
// Let's suppose: The value of the lower limit is 3
(_3D.getObject(1).JointLowerLimit = 3)
// the lower limit is activated
(_3D.getObject(1).JointLowerLimitActive = true)
// the lower limit is not activated
(_3D.getObject(1).JointUpperLimitActive = false)
_3D.getObject(1).JointUpperLimitActive := true
// activates the upper limit and sets it to 3
```

```
// Let's suppose: The lower limit is not activated
(_3D.getObject(1).JointLowerLimitActive = false)
// the upper limit is activate
(_3D.getObject(1).JointUpperLimitActive = false)
_3D.getObject(1).JointLowerLimitActive := true
// activates the upper limit and sets it to 0
```

See also

[_3D.getObject \[SimTalk\]](#)

Lower Limit

[_3D.JointUseShortestRotationDirection \[SimTalk\]](#)

JointType [SimTalk]

Sets the type of joint of the object designated by <Path>.

Remarks

`_3D.getObject` enables you to access animatable objects.

Type

Attribute

Syntax

```
<Path>.JointType:string
```

Assignment Value

You can assign a value of data type *string*.

You can specify "Prismatic joint" or "Revolute joint".

Example

```
MyStation._3D.getObject(1).JointType := "Revolute joint"
```

See also

[_3D.getObject \[SimTalk\]](#)

Type of Joint

JointUpperLimit [SimTalk]

Sets the upper limit of the joint of the object designated by <Path>.

Type

Attribute

Remarks

Plant Simulation exclusively uses limits while **defining poses**. It does not use them to influence any other behavior, such as [_3D.Poses.moveToCoordinate \[SimTalk\]](#), [_3D.Poses.moveToMU \[SimTalk\]](#), etc.

Syntax

```
<Path>.JointUpperLimit: length
```

Assignment Value

The specified value designates:

- Meters for a **Translation**.
For a **Translation** you assign a value of data type *length*.
- Degrees for a **Rotation**. [_3D.getObject](#) enables you to access animatable objects.
For a **Rotation** you assign a value of data type *real*.

Example

```
MyStation._3D.getObject(1).JointUpperLimit := 25
```

See also

[_3D.getObject \[SimTalk\]](#)

[Upper Limit](#)

JointUpperLimitActive [SimTalk]

Sets if the upper limit of the joint of the object designated by <Path> is active (`true`) or not (`false`).

Remarks

`_3D.getObject` enables you to access animatable objects.

Type

Attribute

Syntax

```
<Path>.JointUpperLimitActive:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation._3D.getObject(1).JointUpperLimitActive := true
```

See also

[_3D.getObject \[SimTalk\]](#)

[Upper Limit](#)

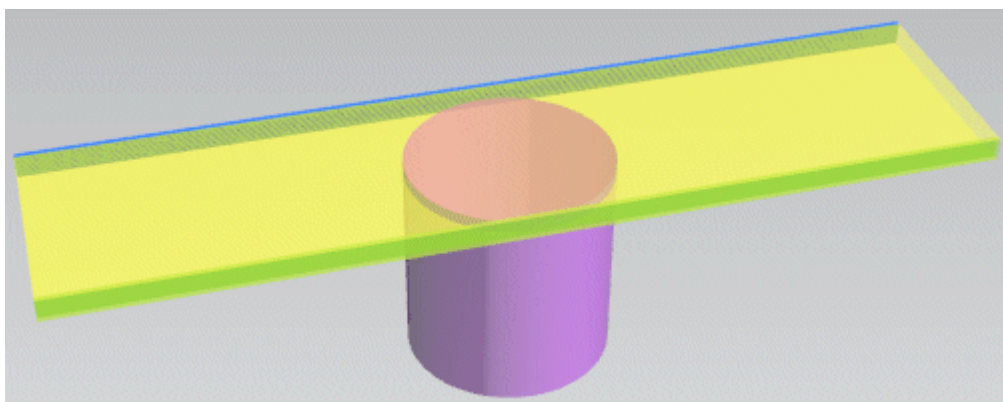
[_3D.JointUseShortestRotationDirection \[SimTalk\]](#)

JointVelocity [SimTalk]

Sets the speed of the joint of the object designated by <Path>.

Remarks

An animatable object with the **Velocity** of 0 does not rotate with the *Turntable* when the *Turntable* rotates. This applies when the insertion position of the animatable object is located on the animation rotation axis of the *Turntable*. This is the case if the insertion point is exactly located on the animation rotation center or is reachable from there along the animation rotation axis without a sideways deviation.



This behavior resembles a real *Turntable* with a fixed base and a rotating table.

`_3D.getObject` enables you to access animatable objects.

Type

Attribute

Syntax

```
<Path>.JointVelocity:speed
```

Assignment Value

The specified value designates:

- The velocity in meters per second for a **Prismatic Joint**.
For a **Prismatic Joint** you assign a value of data type *speed*.
- The angular velocity in degrees per second for a **Revolute Joint**.
For a **Revolute Joint** you assign a value of data type *real*.

Example

```
MyStation._3D.getObject(1).JointVelocity := 25
```

See also

[Velocity \[joint\]](#)

[JointAcceleration \[SimTalk\]](#)

[JointDeceleration \[SimTalk\]](#)

[_3D.getObject \[SimTalk\]](#)

[Turntable \[object\]](#)

moveTo [SimTalk]

Moves to one or several positions or rotations of the joint of the object designated by <Path>.

Remarks

A *moveTo* call of an animatable object aborts a previous *moveTo* run of the same animatable object. For active movements to poses or movements, which the method *_3D.Poses.moveToCoordinate* started, a conflict arises if this animatable object is part of the other active movement.

Note

Whenever you change **Limits**, **Velocity**, **Acceleration**, or the **Joint Type** of the animatable object, the initial position for the next pose run that affects this object returns to 0.

Note

Plant Simulation does not use joint limits in this method. This means, that you can move to a position beyond the limits you specified.

Type

Method

Syntax

```
<Path>.moveTo(PositionOrRotationAngle:real[, TimeSpan:time]) → time
<Path>.moveTo(PositionsOrRotationAngles:array[real][, TimeSpan:time]) →
time
<Path>.moveTo(PositionsOrRotationAngles:array[real][,
TimeSpans:array[time]]) → time
```

Parameters

You can specify the following parameters:

- The parameter *PositionOrRotationAngle* of data type *real* sets the position or the rotation angle of the joint toward which the object moves.

The parameter *PositionsOrRotationAngles* designates an *array* of data type *real* that sets a sequence of positions or rotation angles of the joint to which the object moves. The designated positions or rotations are then approached sequentially one after the other.

The movement stays active until the last position or rotation is reached.

- The optional parameter *TimeSpan* of data type *time* sets the duration of the movement.

The optional parameter *TimeSpans* designates an *array* of data type *time*, which sets the durations of the movement.

Do not specify the parameter to run the pose movement with the time which results from the speeds from the joint definitions of the involved animatable object.

If you specify the parameter, it means the following:

- If you specify a **single time**, the entire movement has to take place within the specified time.
- If you specify a **sequence of times** in an *array*, the first position or rotation is to be reached within the first time span, the second pose in the second time span, etc. You have to specify as many time spans as you defined positions or rotations.

You can only specify an *array of times* if you also specified an *array of positions or rotations*.

The instruction `Station._3D.getObject(1).moveTo(3, [7.5])` results in an error although the same number of positions is specified as the number of time spans. The instruction `Station._3D.getObject(1).moveTo([3], [7.5])` does not result in an error, but is unnecessarily complicated because the instruction `Station._3D.getObject(1).moveTo(3, 7.5)` yields the same result without specifying an array at all.

Return Value

The return value has the data type *time*.

It designates the time span of the simulation time which the designated pose movement will probably take, provided it will not be interrupted.

Example

```
Station._3D.getObject(1).moveTo(3, -1)
Station._3D.getObject(1).moveTo([3, -1], 7.5)
Station._3D.getObject(1).moveTo(3, 7.5)
```

SimTalk

[_3D.Poses.moveTo \[SimTalk\]](#)

[moveToMU \[SimTalk\]](#)

[moveToCoordinate \[SimTalk\]](#)

[moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveToMU \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.AniRotationAxis \[SimTalk\]](#)

[_3D.AniRotationCenter \[SimTalk\]](#)

[_3D.AniTranslationDirection \[SimTalk\]](#)

See also

[Joint](#)

[Open and Close the Gate without Poses](#)

[Move the Gate Up to the Height of the Part](#)

moveToCoordinate [SimTalk]

Moves the joint designated by <Path> to the specified coordinate.

Remarks

When moving a joint to a coordinate, *Plant Simulation* computes the target joint state based upon a relaxed state at which the origin of the joint is located at the position [0,0,0]. The joint points toward the direction specified in the attribute `_3D AniTranslationDirection` in its reset coordinate system.

Note

Plant Simulation does not use joint limits in this method. This means, that you can move to a position beyond the limits you specified.

Type

Method

Syntax

```
<Path>.moveToCoordinate(Coordinate:length[3][, TimeSpan:time]) -> time
```

Parameters

You can specify the following parameters:

- The parameter *Coordinate* is an *array* of data type *length* which specifies the coordinate to which the joint is to move in the joint's coordinate system.

Note

The joint's coordinate system includes its animation state. This also means that if you first compute the coordinate to move to, then move the joint, and then call *moveToCoordinate*, you will end up at another coordinate as the coordinate which you might have intended to.

- The optional parameter *TimeSpan* of data type *time* sets the duration of the movement.

If you do not specify the parameter, *Plant Simulation* runs the movement run with the time which results from the speed from the joint definition of this animatable object.

Return Value

The return value has the data type *time*.

It designates the time span of the simulation time which the designated movement will probably take, provided it will not be interrupted.

Example

```
var destObj = ?.succ
var arm = self.~._3D.getObject("Arm")
var destPos = destObj._3D.getMUAnimationPosition
destPos.z += @.MUHeight
wait arm.moveToCoordinate(arm.getPositionOfObject(destObj, destPos))
```

SimTalk

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

[moveTo \[SimTalk\]](#)

[moveToMU \[SimTalk\]](#)

[moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.moveToMU \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

See also

[Joint](#)

[Video on YouTube](#)

<https://youtu.be/wfVN-mcWNsc?si=Un-c8tYPrM135pgG&t=479>

moveToMU [SimTalk]

Moves the joint designated by <Path> to the specified MU instance.

Remarks

When moving a joint to an MU instance, *Plant Simulation* computes the target joint state based upon a relaxed state at which the origin of the joint is located at the position [0,0,0]. The joint points toward the direction specified in the attribute `_3D.AniTranslationDirection` in its reset coordinate system.

Note

Plant Simulation does not use joint limits in this method. This means, that you can move to a position beyond the limits you specified.

Type

Method

Syntax

```
<Path>.moveToMU(MU:object[, TimeSpan:time]) -> time
```

Parameters

You can specify the following parameters:

- The parameter *MU* of data type *object* designates the MU instance to which the joint moves to.
- The optional parameter *TimeSpan* of data type *time* sets the duration of the movement.

If you do not specify the parameter, *Plant Simulation* executes the movement run with the time which results from the speed from the joint definition of this animatable object.

Return Value

The return value has the data type *time*.

It designates the time span of the simulation time which the designated movement will probably take, provided it will not be interrupted.

Example

```
var mu = otherObject.cont
var arm = self.~._3D.getObject("Arm")
wait arm.moveToMU(mu)
```

SimTalk

[_3D.Poses.moveToMU \[SimTalk\]](#)

[moveTo \[SimTalk\]](#)

[moveToCoordinate \[SimTalk\]](#)

[moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

See also

Joint

moveToMUAnimationPosition [SimTalk]

Moves the joint designated by <Path> to the position and rotation in space where the specified MU instance would be located when being moved to the specified place.

Remarks

When moving a joint to a MU instance, *Plant Simulation* computes the target joint state based upon a relaxed state at which the origin of the joint is located at the position [0,0,0]. The joint points toward the direction specified in the attribute *_3D.AniTranslationDirection* in its reset coordinate system.

If the destination is length-oriented, *moveToMUAnimationPosition* of animatable objects computes a movement, which deposits the front of the MU at the destination instead of the booking point. For place-oriented objects nothing changes.

Note

Plant Simulation does not use joint limits in this method. This means, that you can move to a position beyond the limits you specified.

Type

Method

Syntax

```
<Path>.moveToMUAnimationPosition(MU:object, TargetPE:object/PE[, RelPos/AbsPos:real/length, TimeSpan:time]) -> time
```

Parameters

You can specify the following parameters:

- The parameter *MU* of data type *object* designates the MU instance to measure when moving to the target place.
- The parameter *TargetPE* of data type *object* designates the place to which the item is to move. This can be a specific *PE* (production element) or an *object* if the object only has one PE.
- The optional parameter *RelPos/AbsPos* designates a value of data type *real* or *length*.

If the value has the data type *real*, it designates the relative position on the object. The relative position can be a value between 0 and 1. 0 designates the start of the animation and 1 the end of the animation, i.e., the value 100 %.

If the value has the data type *length*, it designates the absolute position on the object as the distance starting at the beginning of the object.

This parameter will only be used if *TargetPE* specifies a length-oriented place, e.g. a *Conveyor* or a *Transporter* with a length-oriented loadbay.

If you do not specify the parameter, it takes the value 0.0.

- The optional parameter *TimeSpan* of data type *time* designates the duration of the movement.

If you do not specify the parameter, *Plant Simulation* runs the movement run with the time which results from the speed from the joint definition of this animatable object.

Return Value

The return value has the data type *time*.

It designates the time span of the simulation time which the designated movement will probably take, provided it will not be interrupted.

Example

```
var mu = otherObject.cont
var arm = self.~._3D.getObject("Arm")
wait arm.moveToMUAnimationPosition(mu)
```

SimTalk

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[moveTo \[SimTalk\]](#)

[moveToCoordinate \[SimTalk\]](#)

[moveToMU \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.moveToMU \[SimTalk\]](#)

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.getMUAnimationPosition \[SimTalk\]](#)

[_3D.getMUAnimationRotation \[SimTalk\]](#)

See also

[Joint](#)

Accessing Poses

[__Accessing Poses](#)

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for **poses** for objects, which you can insert into a *Frame*, and for *MUs*.

Remarks

In general a **pose** designates a combination of position and orientation of an object. In our context a pose designates a number of translation or rotation positions for animatable objects which belong to a simulation object. This also takes the degrees of freedom into account which you set in the joint definitions of these animatable objects.

A **pose run** designates all movements that were started by any version of the methods *_3D.Poses.moveTo* and *_3D.Poses.moveToCoordinate*, both for the object itself and for any of its animatable objects.

SimTalk

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

See also

[__Accessing Joints](#)

[Joint](#)

[Poses](#)

[Model Joints and Poses](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

Sets if failures and pauses will interrupt **pose runs** of the object designated by <Path> (**true**) or not (**false**).

Remarks

If you start a **pose run** while a failure or pause is active at the moment, the **pose run** will start regardless of the setting *MovementInterruptible*.

A **pose run** designates all movements that were started by any version of the methods *_3D.Poses.moveTo* and *_3D.Poses.moveToCoordinate*, both for the object itself and for any of its animatable objects.

_3D.MovementInterruptible applies to the objects *FluidSource*, *FluidDrain*, *Tank*, *Mixer*, *Transporter*, *Frame*, *Exporter*, and *Worker*. It also applies to all objects which can transport *MUs* or *Workers*, except for the *Workplace*.

Type

Attribute

Syntax

```
<Path>._3D.MovementInterruptible:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
Station._3D.MovementInterruptible := true
```

See also

[Movement Interruptible](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.isMovingTo \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

`_3D.MovementInterruptible [SimTalk]`

`_3D.Poses.moveToCoordinate [SimTalk]`

`_3D.Poses.pauseMovement [SimTalk]`

`_3D.Poses.stopMovement [SimTalk]`

`_3D.Poses.EndPoseWasReached [SimTalk]`

[_3D.Poses.cancelMovement \[SimTalk\]](#)

Cancels the movement toward the designated pose of the object designated by <Path>.

Remarks

If you do not specify the optional parameter, all movements will be canceled.

`_3D.Poses.cancelMovement` affects the designated pose, however always together with the other poses to which the object moved together with this pose in a common call of the method `_3D.Poses.moveTo`.

Type

Method

Syntax

```
<Path>._3D.Poses.cancelMovement([Pose:string])
```

Parameter

The optional parameter *Pose* of data type *string* sets the name of the pose toward which the object moves.

Example

```
Station._3D.Poses.moveTo(["Pose1", "Pose2"])
Station._3D.Poses.cancelMovement("Pose1")

// instead, you can also enter:
Station._3D.Poses.cancelMovement("Pose2")
```

See also

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.stopMovement \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

Continues the movement the designated pose of the object designated by <Path>.

Remarks

If you do not specify the optional parameter, all paused movements will be continued.

`_3D.Poses.continueMovement` affects the designated pose, however always together with the other poses to which the object moved together with this pose in a common call of the method `_3D.Poses.moveTo`.

Type

Method

Syntax

```
<Path>._3D.Poses.continueMovement([Pose:string])
```

Parameter

The optional parameter *Pose* of data type *string* sets the name of the pose which the object moves.

If you do not specify the optional parameter, all paused movements will be continued.

Example

```
Station._3D.Poses.moveTo(["Pose1", "Pose2"])
Station._3D.Poses.continueMovement("Pose1")

// instead, you can also enter:
Station._3D.Poses.continueMovement("Pose2")
```

See also

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

Returns if all **pose runs** for the object designated by <Path> came to an end (true) or not (false).

Remarks

It does not matter if the **pose runs** came to a regular end or if they were canceled with the methods `_3D.Poses.cancelMovement` or `_3D.Poses.stopMovement`.

Pose runs that are only paused by `_3D.Poses.pauseMovement` or by failures or pauses, will not set `_3D.Poses.EndPoseWasReached` to true.

Type

Read-only attribute

Syntax

```
<Path>._3D.Poses.EndPoseWasReached → boolean
```

Watchable

The read-only attribute is [watchable](#).

Return Value

The return value has the data type *boolean*.

Example

```
Station._3D.Poses.moveTo("Pose1")  
waituntil Station._3D.Poses.EndPoseWasReached
```

See also

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.isMovingTo \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

`_3D.Poses.EndPoseWasReached [SimTalk]`

`_3D.Poses.pauseMovement [SimTalk]`

`_3D.Poses.stopMovement [SimTalk]`

[_3D.Poses.getAnimationTime \[SimTalk\]](#)

Returns how long the movement toward the designated pose of the object designated by <Path> takes.

Remarks

If you do not specify the optional parameter, the method returns the simulation time which remains until all pose movements and movements caused by the methods *_3D.Poses.moveToCoordinate* or *moveTo* respectively, are processed, provided that *_3D.Poses.pauseMovement* does not cause a delay.

Type

Method

Syntax

```
<Path>._3D.Poses.getAnimationTime([Pose:string])
```

Parameter

The optional parameter *Pose* of data type *string* sets the name of the pose toward which the object moves.

Example

```
print Station._3D.Poses.getAnimationTime  
print Station._3D.Poses.getAnimationTime("Pose1")
```

SimTalk

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[moveTo \[SimTalk\]](#)

[_3D.Poses.getMUAnimationPosition \[SimTalk\]](#)

Returns the position, which the *MU* would take on the specified PE (production element).

Remarks

This is the same position which would be approached with the method `_3D.Poses.moveToMUAnimationPosition`.

Syntax

```
<Path>(MU:object, TargetPE:any[, RelPos/AbsPos:real/length]) -> real[3]
```

Parameters

You can specify the following parameters:

- The parameter *MU* of data type *object* designates the MU instance to be measured when moving to the target place.
- The parameter *TargetPE* of data type *any* designates the place to which the item is to move. This can be a specific *PE*, or an *object* if the object only has one PE.
- The optional parameter *RelPos/AbsPos* designates a value of data type *real* or *length*.

If the value has the data type *real*, it designates the relative position on the object. The relative position can be a value between 0 and 1. 0 designates the start of the animation and 1 the end of the animation, i.e., the value 100 %.

If the value has the data type *length*, it designates the absolute position on the object as the distance starting at the beginning of the object.

This parameter will only be used if *TargetPE* specifies a length-oriented place, e.g. a *Conveyor* or a *Transporter* with a length-oriented load bay.

If you do not specify the parameter, it takes the value 0.0.

Return Value

The return value is an array with three values of data type *real*.

It designates the X-position, the Y-position, and the Z-position of the *MU* on the PE.

Examples

```
var dropPos = Robot._3D.Poses.getMUAnimationPosition(@,  
ParallelStation[1,2])
```

See also

[_3D.Poses.getMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.getTable \[SimTalk\]](#)

Returns the poses of the object designated by <Path> and writes them into the specified table.

Type

Method

Syntax

```
<Path>._3D.Poses.getTable(Poses:table)
```

Parameter

The parameter *Poses* of data type *table* designates the table in which the data of the poses are going to be written.

Example

```
var t : table  
MyStation._3D.Poses.getTable(t)  
MyStation._3D.Poses.setTable(t)
```

See also

[_3D.Poses.setTable \[SimTalk\]](#)

[_3D.Poses.isMovingTo \[SimTalk\]](#)

Returns if the object designated by <Path> moves toward the designated pose.

Remarks

[_3D.Poses.isMovingTo](#) affects the designated pose, however always together with the other poses to which the object moved together with this pose in a common call of the method [_3D.Poses.moveTo](#).

Type

Method

Syntax

```
<Path>._3D.Poses.isMovingTo(Pose:string) → boolean
```

Parameter

The parameter *Pose* of data type *string* sets the name of the pose toward which the object moves.

Return Value

The return value has the data type *boolean*.

true if the object moves toward the pose.

false if the object does not move toward the pose.

If the pose name is either empty or does not match any existing pose, the method returns an error.

Example

```
print Station._3D.Poses.isMovingTo("Pose1")
```

See also

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

Moves to one or several poses of the object designated by <Path>.

Remarks

Whenever you change the [Lower Limit/upper Limit](#), the [Velocity](#), the [Acceleration](#), or the [Joint Type](#) of an animatable object, the initial position for the next pose run that affects this object returns to 0.

When all pose runs of an object reached their destination, *Plant Simulation* enters an **EndOfTime** event into the [Event List](#).

Type

Method

Syntax

```
<Path>._3D.Poses.moveTo(Pose:string[, TimeSpan:time]) → time  
<Path>._3D.Poses.moveTo(Poses:array[string][, TimeSpan:time]) → time  
<Path>._3D.Poses.moveTo(Poses:array[string][, TimeSpans:array[time]]) →  
time
```

Parameters

You can specify the following parameters:

- The parameter *Pose* of data type *string* sets the name of the pose toward which the object moves.

The parameter *Poses* designates an *array* of data type *string* containing the names of several consecutive poses. The object moves to the designated poses sequentially one after the other.

If you specify invalid parameters or if you specify a pose that contains an animatable object that is part of a movement that is still active, the method opens the *Method Debugger* with an error. In this context a pose movement always consists of all poses that are specified together with the method `_3D.Poses.moveTo`. It stays active until the last pose is reached.

If the object is to move to a pose named "init" and no pose with that name exists, the object moves to an implicit pose which sets all animatable objects of the object designated by <Path> to 0.

- The optional parameter *TimeSpan* of data type *time* sets the duration of the movement.

The optional parameter *TimeSpans* designates an *array* of data type *time*, which sets the durations of the movement.

If you do not specify the parameter, the pose movement is run with the time which results from the speeds from the joint definitions of the involved animatable objects.

If you specify the parameter, it means the following:

- If you specify a **single time**, the entire pose movement has to take place within the specified time. The time relations of the joints and the individual pose movements among each other are retained. If the instruction `Station._3D.Poses.moveTo(["Pose1", "Pose2"])` takes 3 seconds, of which 1 second is allotted to Pose1 and 2 seconds are allotted to Pose2, then the instruction `Station._3D.Poses.moveTo(["Pose1", "Pose2"], 7.5)` takes 7.5 seconds of which 2.5 seconds are allotted to Pose1 and 5 seconds are allotted to Pose2.
- If you specify a **sequence of times** in an *array*, the first pose is to be reached within the first time span, the second pose in the second time span, etc. You have to specify as many time spans as you defined poses.

The instruction `Station._3D.Poses.moveTo(["Pose1", "Pose2"], 7.5)` from the example above and the instruction `Station._3D.Poses.moveTo(["Pose1", "Pose2"], [2.5, 5])` thus are equivalent. You can only specify an *array of times* if you also specified an *array of poses*.

The instruction `Station._3D.Poses.moveTo("Pose1", [7.5])` results in an error although the same number of poses is specified as the number of time spans. The instruction `Station._3D.Poses.moveTo(["Pose1"], [7.5])` does not result in an error, but is unnecessarily complicated because the instruction `Station._3D.Poses.moveTo("Pose1", 7.5)` yields the same result without specifying an *array* at all.

Return Value

The return value has the data type *time*.

It designates the time span of the simulation time which the designated pose movement will probably take, provided it will not be interrupted.

Example

```
Station._3D.Poses.moveTo(["Pose1", "Pose2"])
Station._3D.Poses.moveTo(["Pose1", "Pose2"], 7.5)
Station._3D.Poses.moveTo("Pose1", 7.5)
```

SimTalk

[moveTo \[SimTalk\]](#)

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

[_3D.Poses.moveToMU \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[moveToCoordinate \[SimTalk\]](#)

[moveToMU \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.isMovingTo \[SimTalk\]](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.AniRotationAxis \[SimTalk\]](#)

[_3D.AniRotationCenter \[SimTalk\]](#)

[_3D.AniTranslationDirection \[SimTalk\]](#)

See also

[Joint Settings of an Animatable Object](#)

[Open and Close the Gate with Poses](#)

Video on YouTube

<https://youtu.be/rQi5oRyZckQ?si=pGkDwvoDw3LOD4Wo&t=1130>

<https://youtu.be/wfVN-mcWNsc?si=StxMHuDgvZerba9g&t=269>

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

Moves the object designated by <Path> to the specified coordinate with its first animation point to the designated coordinate and to the designated rotation as long as the identified robot configuration supports this. This works without you having to define a pose.

Remarks

_3D.Poses.moveToCoordinate does not support the four axis robot with ball joint gripper. The object corresponds to one of the supported robot configurations.

Plant Simulation checks the attribute *_3D.AnimationObject* to identify the robot configuration. The objects, which are located between the simulation object, which is excluded, and *_3D.AnimationObject*, which is included, are checked for the appropriate robot configuration with which the pose movement is executed.

Plant Simulation checks if a part of the axes configuration matches a supported robot type from the simulation object level to the gripper object. A robot type with more axes takes precedence over a robot type with fewer axes.

Note

If *Plant Simulation* cannot identify the robot configuration, this is considered an error and the *Method Debugger* will open.

Note

Not reaching the destination coordinate/rotation is not considered an error and does not open the *Method Debugger*. An unsuitable, but recognizable axes configuration will not be recognized, if eight axes exist, for example, but only the last three axes are recognized as part of a simpler robot type.

Note

Plant Simulation does not use joint limits in this method. This means, that you can move to a position beyond the limits you specified.

Type

Method

Syntax

```
<Path>._3D.Poses.moveToCoordinate(Coordinate:length[3], Rotation:real/  
real[4][, TimeSpan:time]) -> time
```

Parameters

You can specify the following parameters:

- The parameter *Coordinate* is an *array* of data type *length* with three values which designates the destination coordinate in the coordinate system of the object to be moved.
- The parameter *Rotation* is a value of data type *real* which designates a rotation around the negative z-axis.

You can also specify the parameter *Rotation* as an *array* of data type *real* with four values. It designates a rotation, compare *_3D.Rotation*, which the animation point of the object to be moved is to take within its coordinate system after finishing the pose movement.

- The optional parameter *TimeSpan* of data type *time* sets the duration of the movement.

If you do not specify the parameter, *Plant Simulation* runs the movement run with the time which results from the speed from the joint definition of this animatable object.

Return Value

The return value has the data type *time*.

It designates the time span of the simulation time which the designated movement will probably take, provided it will not be interrupted.

Examples

```
Station._3D.Poses.moveToCoordinate([1m, 2m, 0m], 0)
Station._3D.Poses.moveToCoordinate([1m, 2m, 0m], 45, 1:00)
Station._3D.Poses.moveToCoordinate([1m, 2m, 0m], [90, 0, 1, 0])
```

```
Station._3D.Poses.moveToCoordinate(Station._3D.getPositionFromObject(OtherStations), 0)
```

```
var Rotation = Station._3D.getRotationFromObject(OtherObject)
Rotation :=
F3DconcatenateRotations(Station._3D.getRotationFromObject(OtherObject),
[180, 0, 1, 0])
Station._3D.Poses.moveToCoordinate(Station._3D.getPositionFromObject(OtherObject), Rotation)
// grip 'OtherObject' from above
```

SimTalk

[moveToCoordinate \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.moveToMU \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[moveTo \[SimTalk\]](#)

[moveToMU \[SimTalk\]](#)

[moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.AnimationObject \[SimTalk\]](#)

[_3D.Rotation \[SimTalk\]](#)

See also

[Configuring the Robot](#)

[_3D.Poses.moveToMU \[SimTalk\]](#)

Moves the object designated by <Path> with its first animation point to the specified MU instance as long as the identified robot configuration supports this. This works without you having to define a pose.

Remarks

`_3D.Poses.moveToMU` does not support the **four axis robot with ball joint gripper**. The object corresponds to one of the supported robot configurations.

Plant Simulation checks the attribute `_3D.AnimationObject` to identify the robot configuration. The objects, which are located between the simulation object, which is excluded, and `_3D.AnimationObject`, which is included, are checked for the appropriate robot configuration with which the pose movement is executed.

Plant Simulation checks if a component of the axes configuration matches a supported robot type from the simulation object level to the gripper object. A robot type with more axes takes precedence over a robot type with fewer axes.

Note

If *Plant Simulation* cannot identify the robot configuration, this is considered an error and the *Method Debugger* will open.

Note

Not reaching the destination MU instance is not considered an error and does not open the *Method Debugger*. An unsuitable, but recognizable axes configuration will not be recognized, if eight axes exist, for example, but only the last three axes are recognized as part of a simpler robot type.

Type

Method

Syntax

```
<Path>_3D.Poses.moveToMU(MU:object[, TimeSpan:time]) -> time
```

Parameters

You can specify the following parameters:

- The parameter *MU* of data type *object* designates the MU instance to which the object is to move.
- The optional parameter *TimeSpan* of data type *time* sets the duration of the movement.

If you do not specify the parameter, *Plant Simulation* executes the movement run with the time which results from the speed from the joint definition of the involved animatable objects.

Return Value

The return value has the data type *time*.

It designates the time span of the simulation time which the designated movement will probably take, provided it will not be interrupted.

Examples

```
var mu = otherObject.cont
var arm = self._3D.getObject("Arm")
wait arm.moveToMU(mu)
```

```
-- control MyRob action to pick parts at the store
-- and drop them on the station
var nextPart : object

for var i := 1 to 12
    nextPart = Store.cont
    waituntil station.empty

    -- move to part and load it
    wait MyRob._3D.Poses.MoveToMU(nextPart)
    nextPart.move(MyRob)
    wait 1

    -- move to target station and drop part there
    wait MyRob._3D.Poses.MoveToMUAnimationPosition(nextPart, Station)
    nextPart.move(Station)
    wait 1

    -- move robot to home position
    wait MyRob._3D.Poses.moveTo("home")
next
```

SimTalk

[moveToMU \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveToMU \[SimTalk\]](#)

[moveTo \[SimTalk\]](#)

[moveToCoordinate \[SimTalk\]](#)

[moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.AnimationObject \[SimTalk\]](#)

See also

Joint

_3D.Poses.moveToMUAnimationPosition [SimTalk]

Moves the object designated by <Path> with its first animation point to the position and rotation in space where the specified MU instance would be located when being moved to the specified place as long as the identified robot configuration supports this. This works without you having to define a pose.

Remarks

_3D.Poses.moveToMUAnimationPosition does not support the **four axis robot with ball joint gripper**. The object corresponds to one of the supported robot configurations.

Plant Simulation checks the attribute *_3D.AnimationObject* to identify the robot configuration. The objects, which are located between the simulation object, which is excluded, and *_3D.AnimationObject*, which is included, are checked for the appropriate robot configuration with which the pose movement is executed.

Plant Simulation checks if a part of the axes configuration matches a supported robot type from the simulation object level to the gripper object. A robot type with more axes takes precedence over a robot type with fewer axes.

If the destination is length-oriented *_3D.Poses.moveToMUAnimationPosition* of animatable objects computes a movement, which deposits the front of the MU at the destination instead of the booking point. For place-oriented objects nothing changes.

Note

If *Plant Simulation* cannot identify the robot configuration, this is considered an error and the *Method Debugger* will open.

Note

Not reaching the destination place is not considered an error and does not open the *Method Debugger*. An unsuitable, but recognizable axes configuration will not be recognized, if eight axes exist, for example, but only the last three axes are recognized as part of a simpler robot type.

Note

Plant Simulation does not use joint limits in this method. This means, that you can move to a position beyond the limits you specified.

Type

Method

Syntax

```
<Path>_3D.Poses.moveToMUAnimationPosition(MU:object, TargetPE:object/PE[,  
RelPos/AbsPos:real/length, TimeSpan:time]) -> time
```

Parameters

You can specify the following parameters:

- The parameter *MU* of data type *object* designates the MU instance to be measured when moving to the target place.
- The parameter *TargetPE* of data type *object* designates the place to which the item is to move. This can be a specific *PE* (production element), or an *object* if the object only has one PE.
- The optional parameter *RelPos/AbsPos* designates a value of data type *real* or *length*.

If the value has the data type *real*, it designates the relative position on the object. The relative position can be a value between 0 and 1. 0 designates the start of the animation and 1 the end of the animation, i.e., the value 100 %.

If the value has the data type *length*, it designates the absolute position on the object as the distance starting at the beginning of the object.

This parameter will only be used if *TargetPE* specifies a length-oriented place, e.g. a *Conveyor* or a *Transporter* with a length-oriented load bay.

If you do not specify the parameter, it takes the value 0.0.

- The optional parameter *TimeSpan* of data type *time* designates the duration of the movement.
If you do not specify the parameter, the movement is run with the time which results from the speeds from the joint definitions of the involved animatable objects.

Return Value

The return value has the data type *time*.

It designates the time span of the simulation time which the designated movement will probably take, provided it will not be interrupted.

Examples

```
var mu = otherObject.cont  
wait self.~._3D.moveToMUAnimationPosition(mu)
```

```
-- control MyRob action to pick parts at the store  
-- and drop them on the station  
var nextPart : object
```

```
for var i := 1 to 12
  nextPart = Store.cont
  waituntil station.empty

  -- move to part and load it
  wait MyRob._3D.Poses.MoveToMU(nextPart)
  nextPart.move(MyRob)
  wait 1

  -- move to target station and drop part there
  wait MyRob._3D.Poses.MoveToMUAnimationPosition(nextPart, Station)
  nextPart.move(Station)
  wait 1

  -- move robot to home position
  wait MyRob._3D.Poses.moveTo("home")
next
```

SimTalk

[moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.moveToMU \[SimTalk\]](#)

[_3D.Poses.moveToCoordinate \[SimTalk\]](#)

[moveTo \[SimTalk\]](#)

[moveToCoordinate \[SimTalk\]](#)

[moveToMU \[SimTalk\]](#)

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.Poses.getMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.getMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.getMUAnimationRotation \[SimTalk\]](#)

[_3D.AnimationObject \[SimTalk\]](#)

See also

[Configuring the Robot](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

Pauses the movement toward the designated pose of the object designated by <Path>.

Remarks

If you do not specify the optional parameter, all movements will be paused.

`_3D.Poses.pauseMovement` affects the designated pose, however always together with the other poses to which the object moved together with this pose in a common call of the method `_3D.Poses.moveTo`.

Type

Method

Syntax

```
<Path>._3D.Poses.pauseMovement([Pose:string])
```

Parameter

The optional parameter *Pose* of data type *string* sets the name of the pose toward which the object moves.

Example

```
Station._3D.Poses.moveTo(["Pose1", "Pose2"])
Station._3D.Poses.pauseMovement("Pose1")

// instead, you can also enter:
Station._3D.Poses.pauseMovement("Pose2")
```

See also

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.continueMovement \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)

[_3D.Poses.setTable \[SimTalk\]](#)

Sets the poses of the object designated by <Path> from the specified table.

Type

Method

Syntax

```
<Path>._3D.Poses.setTable(Poses:table)
```

Parameter

The parameter *Poses* of data type *table* designates the table which contains the data of the poses.

Example

```
var t : table  
MyStation._3D.Poses.getTable(t)  
MyStation._3D.Poses.setTable(t)
```

See also

[_3D.Poses.getTable \[SimTalk\]](#)

[_3D.Poses.stopMovement \[SimTalk\]](#)

Stops the movement toward the designated pose of the object designated by <Path> and, if you entered a deceleration for the joints, decelerates toward a halt.

Remarks

Do not specify the optional parameter to stop all movements.

_3D.Poses.stopMovement affects the designated pose, however always together with the other poses to which the object moved together with this pose in a common call of the method *_3D.Poses.moveTo*.

Remarks

Type

Method

Syntax

```
<Path>._3D.Poses.stopMovement([Pose:string]) -> time
```

Parameter

The optional parameter *Pose* of data type *string* sets the name of the pose toward which the object moves.

Return Value

The return value has the data type *time*.

Example

```
Station._3D.Poses.moveTo(["Pose1", "Pose2"])
wait Station._3D.Poses.stopMovement("Pose1")

// instead, you can also enter:
wait Station._3D.Poses.stopMovement("Pose2")
```

See also

[_3D.Poses.cancelMovement \[SimTalk\]](#)

[_3D.Poses.moveTo \[SimTalk\]](#)

[_3D.Poses.pauseMovement \[SimTalk\]](#)

[_3D.MovementInterruptible \[SimTalk\]](#)

[_3D.Poses.EndPoseWasReached \[SimTalk\]](#)